arXiv:2604.17538v2 [cs.RO] 24 May 2026

Novel Algorithms for Smoothly Differentiable and Efficiently Vectorizable Contact Manifold Construction

Onur Beker, A. René Geist, Anselm Paulus, Georg Martius,
University of Tübingen

***Abstract*— Generating intelligent robot behavior in contact-rich settings is a research problem where zeroth-order methods currently prevail. Developing methods that make use of first/second order information about rigid-body dynamics in the presence of contact holds great promise in terms of increasing the solution speed and computational efficiency. The main bottleneck in this research direction is the difficulty in obtaining gradients and Hessians that are actually useful for numerical optimization, due to pathologies in all three steps of a common simulation pipeline: i) collision detection, ii) contact dynamics, iii) time integration. This abstract proposes a method that aims to address the collision detection part of the puzzle, via a novel pipeline designed from scratch with smooth (i.e. twice) differentiability and massive vectorizability on GPUs as the main priorities. This is in contrast to standard collision detection routines that are instead optimized for runtime on CPUs and minimal memory footprint, but do employ logic and control flow that hinder differentiability and vectorization. The proposed pipeline consists of the following contributions: i) highly expressive and compute efficient SDF representations, ii) differentiable broad-phase and narrow-phase routines that use these representations to generate vertex-SDF and edge-SDF contacts, iii) a differentiable routine for convex decomposition based contact blending.**

I. Introduction and Related Work

Our exposition proceeds as follows. We first briefly describe how existing collision detection pipelines based on convex primitive decomposition work, which are employed as the de-facto standard by all commonly used simulators in robotics [1, 2, 3]. We then outline existing approaches for making such convex primitive based routines smoothly differentiable. Finally, we list the reasons why such approaches still fall short of providing a satisfactory, generally applicable solution to differentiable collision detection.

The standard non-differentiable process employed by commonly used simulators to generate a contact manifold (i.e. a properly distributed set of contact points) between two surfaces can be abstracted into the following steps [4, 5]:

- Converting each non-convex surface into a mesh with well-distributed faces using a computational geometry algorithm [6, 7, 8], and then decomposing it into a set of convex meshes [9]. All of the remaining steps operate on pairs of these convex meshes.
- Running standard collision detection routines like GJK+EPA [10, 11, 12] or SAT [13] to obtain information such as a pair of "witness points" (the end-points of the largest penetration distance) or closest mesh facets.
- Building a contact manifold that sufficiently represents the entire volume/area of intersection between surfaces.

*An earlier version of this manuscript is accepted for presentation at the ICRA 2026 Workshop on Contact-Rich Control and Representation. The present document contains late-breaking additions in preparation for the workshop presentation.

The last step of going from two witness points to a manifold is needed to stabilize simulations in pathological settings such as completely parallel faces (e.g. two boxes stacked on top of each other). Alternative ways of implementing it are [5]:

- Incrementally building it across simulation time-steps by maintaining a buffer of witness points based on distance heuristics about when to add or remove from the buffer.
- Building a contact manifold from scratch every time-step (i.e. "one-shot") by: i) running GJK+EPA to identify the penetration vector and tangential axes to it, and ii) inducing small rotations on the two geometries multiple times around the tangential axes and re-running GJK+EPA each time.
- Building a contact manifold one-shot by applying polygon clipping algorithms [14] to the closest mesh-facets.

Approaches for obtaining smoothed gradients for the witness point positions between two convex primitives can be grouped in two: analytical smoothing and randomized smoothing. A prominent example of the former is the work of Tracy et al. [15], which solves for the growth distance [16] between pairs of a representative set of convex primitives (including convex meshes) by formulating them as cone-constrained convex programs. Gradients through this convex program are then obtained via implicit differentiation and smoothed via modulating the log-barrier coefficient of the underlying interior-point solver [17]. A prominent example of the randomized smoothing approach is the work by Montaut et al. [18], which operates by: i) sampling M deviations around a nominal kinematic configuration, ii) running GJK and EPA algorithms [10] M times to get separation distances, iii) getting a zeroth-order estimate of the gradient via the score function estimator [19]. While both methods can robustly generate useful gradients for the witness point positions between two convex primitives, we argue that the underlying convex primitive based paradigm itself is pathological for differentiable contact simulation, as we elaborate on next.

The first pathology of the convex primitive based formulation is that witness points are conceptually not well defined (i.e. non-unique) for configurations that involve parallel contacts between planar faces, which are very common in contact simulations since they often constitute stable points for the dynamics (e.g. a box on a plane). For the analytical smoothing approach, this means in such pathological configurations (where the non-smoothed witness point optimization problem is not strictly convex), the uniqueness comes from the (strictly convex) log-barrier term, which needs to be sufficiently flat for the gradients to remain physically plausible, thus leading to slow convergence. For the randomized smoothing approach, this means the estimator has large variance at such

configurations, since witness points can jump drastically in arbitrary directions based on the sampled deviation. The second pathology is that a single witness points pair is not sufficient for stable simulation, and only addressing the differentiability of the convex-convex witness point routine is therefore insufficient. For the subsequent step of building a contact manifold from witness points, maintaining a buffer couples gradients across timesteps which is unsuitable for optimal control, and making polygon clipping routines differentiable in a computationally efficient manner is non-trivial (due to branching code and non-smooth operations such as top-K selection) [20] and remains an open problem. The most commonly used option is therefore inducing incremental rotations along tangential axes and re-running the convex-convex routine to build a manifold. The main problem is that there is no standard, mathematically principled way of picking the tangential axes uniquely (since they are only defined up to a rotation along the normal direction), in a way that is guaranteed to change smoothly across timesteps. A third and final pathology is that in the absence of a broad-phase routine for filtering primitive pairs without running the complete convex-convex routine, the convex primitive based formulation is inefficient. This is because for two surfaces decomposed into N and M primitives each, $N \times M$ convex-convex checks are required. Therefore only addressing the narrow-phase convex-convex routine is insufficient since the broad-phase routine can still create jumps in the gradient (due to primitive pairs activating and deactivating).

All of these considerations motivate us to design a collision detection routine from scratch with differentiability and vectorizability as the primary concerns (which constitutes the main contribution of this paper), inspired by the analogous routines of barrier-based contact simulation approaches [21]. We opt for this approach rather than trying to make existing routines differentiable, because these standard routines were conceived within a computer graphics, video game design, or computational science and engineering context with speed (i.e. of a single simulation instance on CPU) and minimal memory footprint as the main priorities.

II. Background: Smooth Approximations to Non-Smooth Operators

The proposed framework extensively uses smooth approximations of non-smooth operators such as comparisons and clipping. This section therefore aims to provide a highly compressed summary for completeness. A much more detailed exposition can be found in the SoftJAX library [22].

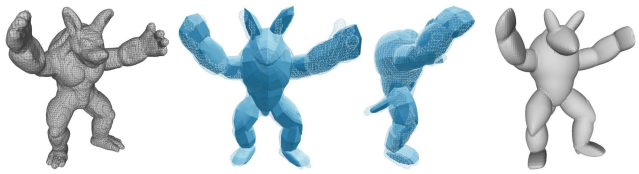
The sigmoid function $\sigma(x) = (1+\exp(-x))^{-1}$ can be used to smoothly approximate a comparison operator $[\![x > a]\!] : \mathbb{R} \to [0,1]$ (notated with an Iverson-Bracket) via $\sigma((x-a)/\tau)$. The $\text{relu}(x)$ function, which projects a number onto the positive line-segment, can be smoothly approximated using a softplus function $\text{s}_+(x) = \tau \log(1+\exp(x/\tau))$. The clipping operation $f_{\text{clip}}(x, x_{\min}, x_{\max}) = x_{\min} + \text{relu}(x - x_{\min}) - \text{relu}(x - x_{\max})$ can in turn be softened into a softclip operation $s_{\text{clip}}(x, x_{\min}, x_{\max})$ by replacing relu operations with s_+. The scalar sigmoid and softplus functions can be generalized to operate on vectors $\boldsymbol{x} \in \mathbb{R}^D$ to obtain the softmax and realsoftmax (i.e. logsumexp) functions respectively, notated as $s_{\text{argmax}}(\boldsymbol{x})_i = \exp(x_i/\tau)/\sum_{j=1}^{D}\exp(x_j/\tau)$ and $\text{LSE}(\boldsymbol{x}) = \tau \log[\sum_{i=1}^{D}\exp(x_i/\tau)]$. Note that $\boldsymbol{s}_{\text{argmax}}(\boldsymbol{x}) \in \mathbb{R}^D$ is a vector of probability masses that approximates an argmax operation, whereas $\text{LSE}(\boldsymbol{x}) \in \mathbb{R}$ is a scalar that corresponds to a soft selection of the maximum component of $\boldsymbol{x}$.

III. Methods: Surface Representations

In the proposed framework, collisions happen between one surface represented with a signed distance field (SDF), and the vertices and edges of another surface that is represented with a mesh. Symmetry is achieved by simply running the routine again with these representation type assignments (i.e. SDF or mesh) transposed. Therefore, the smooth differentiability of the entire routine relies on constructing SDFs that are smoothly differentiable functions, which is the focus of this section. The proposed framework provides two alternative ways of constructing such SDFs to suit different needs: i) as a superposition of analytical primitives called "extruded plane-superquadric intersections" (XPSQ) , ii) as 3D splines (which themselves are superpositions of cubic polynomial basis functions). The benefits of the former primitive-based approach are that: i) it is very memory efficient, and ii) it easily admits randomized alterations in a way that preserves surface semantics (e.g. for procedural generation or domain randomization); but has the downside that it needs to be constructed by hand for each new surface since no automated decomposition method yet exists that can read a mesh and return a list of XPSQs that approximate it. The latter spline-based approach is less memory efficient and harder to randomize, but can automatically be constructed from arbitrary meshes and is therefore easier to work with.

A. Primitive-Based SDF Representation

This section elaborates on the proposed "extruded plane-superquadric intersection" (XPSQ) primitive, extending the line of work by [23, 24], in particular on the superquadric (SQ) primitive. At a high-level, the XPSQ primitive analytically solves for the SDF associated with the volume traced by sweeping a superquadric (potentially intersected with N half-spaces) along a 1D quadratic spline curve embedded in 3D. This enables describing geometries such as the handle of a cup with a single primitive, which would not be possible via convex decomposition [9] or an SQ decomposition [25].



Original Mesh SQ Decomposition SDF Isosurface

Fig. 1: An armadillo geometry approximated as the smooth union of 18 superquadric (SQ) SDFs, illustrating the highly expressive nature of SQ primitives that from the foundation of XPSQ primitives.

1) Superquadrics. Superquadrics (SQ) are a family of surfaces that subsume basic shape primitives such as cubes, cylinders, and ellipsoids. An SQ in its canonical form is

defined by the unit level set the level set $f(\boldsymbol{x}) = 1$ of an *inside–outside* implicit function $f : \mathbb{R}^3 \to \mathbb{R}$ [25]:

$$f(\mathbf{x}) = \left(\left(\frac{x}{a_x} \right)^{\frac{2}{\varepsilon_2}} + \left(\frac{y}{a_y} \right)^{\frac{2}{\varepsilon_2}} \right)^{\frac{\varepsilon_2}{\varepsilon_1}} + \left(\frac{z}{a_z} \right)^{\frac{2}{\varepsilon_1}} \tag{1}$$

where ε_1 controls the shape's "roundness", ε_2 adjusts the shape's "pointedness", and a_x scales the x-dimension. In total there are 11 parameters: 6 for 3D pose, 3 for per axis scales, 2 for roundness and pointedness.

2) Differentiable Combinations of SDFs. Any two SDFs $\phi_1(\boldsymbol{x}), \phi_2(\boldsymbol{x}) : \mathbb{R}^3 \to \mathbb{R}$ (e.g. representing a half space, SQ, or XPSQ) can be combined through *union* $\phi_3 = \min(\phi_1, \phi_2)$, *intersection* $\phi_3 = \max(\phi_1, \phi_2)$, or *subtraction* $\phi_3 = \max(\phi_1, -\phi_2)$ operations. Smoothly differentiable versions of these operations are obtained through the use of the logsumexp operator [26, 27]:

$$\phi_\cup(\boldsymbol{x}) = -\text{LSE}([-\phi_1(\boldsymbol{x}), -\phi_2(\boldsymbol{x})]) \quad \text{(smooth union)}, \tag{2}$$

$$\phi_\cap(\boldsymbol{x}) = \text{LSE}([\phi_1(\boldsymbol{x}), \phi_2(\boldsymbol{x})]) \quad \text{(smooth intersection)}, \tag{3}$$

$$\phi_\ominus(\boldsymbol{x}) = \text{LSE}(\phi_+(\boldsymbol{x}), -\phi_-(\boldsymbol{x})) \quad \text{(smooth subtraction)}. \tag{4}$$

3) PSQ: Combining Half Spaces with SQs. The SDF of a half-space is defined as $\phi_N = \boldsymbol{x} \cdot \boldsymbol{n} + h$, where $\boldsymbol{n}$ denotes the plane normal and h the offset from the origin. Through the smooth intersection (3) of an SQ and a single half space, most of the geometric primitives provided by common simulation frameworks can be defined, including boxes, ellipsoids, cylinders, elliptic cones, square pyramids, and tetrahedrons (although our framework and software implementation does not assume any restrictions on the number of half spaces intersecting an SQ). We refer to the primitive obtained by intersecting N half spaces $\mathbf{P} \in \mathbb{R}^{N\times 4}$ (3 parameters for $\boldsymbol{n}$ and 1 for h) and an SQ as a "plane-SQ intersection" (PSQ).

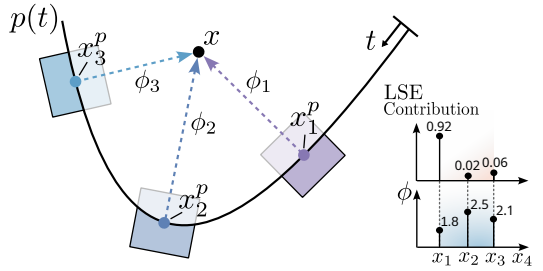


Fig. 2: The SDF of the XPSQ primitive is evaluated by: (i) analytically projecting a point $\boldsymbol{x}$ onto a spline $p(t)$, (ii) transforming the PSQ SDF expression such that the surface is moved to coincide with each of the three projection points, and (iii) obtaining the SDF for $\boldsymbol{x}$ via the smooth-minimum of the three transformed PSQ SDFs.

4) XPSQ: Tracing a Spline with a PSQ. The XPSQ primitive represents the volume obtained by tracing a PSQ along a 1D quadratic spline curve embedded in 3D. Such a quadratic spline curve admits a parameterization using $t \in [0,1]$ and three control points $\boldsymbol{p}_{1:3}$ as follows:

$$\boldsymbol{p}(t) = (1-t)^2\boldsymbol{p}_1 + 2t(1-t)\boldsymbol{p}_2 + t^2\boldsymbol{p}_3. \tag{5}$$

The data-structure that represents an XPSQ primitive therefore maintains the following information: i) a quadratic spline $\boldsymbol{p}(t)$, ii) a smooth function $\mathbf{R}(t) \in \text{SO}(3)$ that prescribes the rotational pose of the PSQ as it traces the spline (e.g. the Frenet frame associated with the spline [28]), iii) smooth functions $\varepsilon_{\{1,2\}}(t)$ and $a_{\{x,y,z\}}(t)$ that prescribe the shape and scale parameters of the SQ part of the PSQ, iii) $\mathbf{P}(t) \in \mathbb{R}^{N\times 4}$ that prescribes the plane normal (expressed in the PSQ base frame) and shift for the N half spaces within the PSQ.

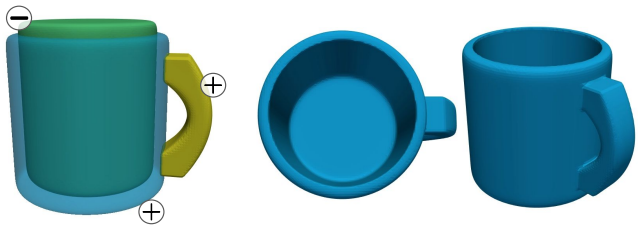


Fig. 3: Combining XPSQ primitives via SDF union, intersection, and subtraction operations provides an efficient and highly expressive way to represent SDFs of challenging surfaces, such as a cup.

5) The SDF Associated with an XPSQ. To evaluate the SDF of the XPSQ for a given point $\boldsymbol{x}$, one first projects it onto the spline to obtain the projection point $\boldsymbol{p}(t^*)$, and then evaluates the SDF of the PSQ at that point (as prescribed by $\mathbf{R}(t^*), \varepsilon_{\{1,2\}}(t^*), a_{\{x,y,z\}}(t^*), \mathbf{P}(t^*)$). As was also noted by [29], this projection can be obtained by minimizing $\|\boldsymbol{x} - \boldsymbol{p}(t)\|^2$, which is a fourth order polynomial in t. Taking its gradient and setting it to zero in turn gives a cubic equation in t, which can be solved analytically via Cardano's formula [30]. The solution is not unique in general, as for some constellations there are multiple points on the spline with equally minimal distance to $\boldsymbol{x}$, e.g. when the spline is wrapped around $\boldsymbol{x}$. In Cardano's formula, these cases are distinguished by the sign of the discriminant $\Delta \in \mathbb{R}$, which is a function of the control points, and the computation of the roots branches depending on it. This branching leads to a discontinuity, which we soften by always treating both branches simultaneously and merging the results. In the negative branch we use the negative projected discriminant $\Delta^- = -s_+(-\Delta) \in \mathbb{R}^-$ to obtain the unique real root t^- of the cubic. In the positive branch, we use the positive projected discriminant $\Delta^+ = s_+(\Delta) \in \mathbb{R}^+$, yielding three real roots (t_1^+, t_2^+, t_3^+). Note that in both cases we apply a soft clipping operation to keep the roots in the range $(0,1)$, as Cardano's formula can return roots outside of this range. Next, we always return three roots as the convex combination $\boldsymbol{t}^* \in \mathbb{R}^3$ of the two root sets:

$$\boldsymbol{t}^* = [\![\Delta < 0]\!] * [t^-, t^-, t^-] + [\![\Delta > 0]\!] * [t_1^+, t_2^+, t_3^+] \tag{6}$$

where the brackets are smoothed as described in S.II. Then, we get the PSQs prescribed at all three roots using $\mathbf{R}(t_i^*)$, $\boldsymbol{x}_i^*$, $\varepsilon_{\{1,2\}}(t_i^*)$, $a_{\{x,y,z\}}(t_i^*)$, and $\mathbf{P}(t_i^*)$ with $i \in \{1,2,3\}$. The associated SDFs for the three PSQs are $\{\phi(\boldsymbol{x}_1^*), \phi(\boldsymbol{x}_2^*), \phi(\boldsymbol{x}_3^*)\}$, and we finally return their smooth-minimum $\phi_\cup(\boldsymbol{x}) = -\text{LSE}([-\phi_1(\boldsymbol{x}), -\phi_2(\boldsymbol{x}), -\phi_3(\boldsymbol{x})])$ as shown in Fig.2.

B. Spline-Based SDF Representation

This section provides a short background on how 3D cubic splines that map $\mathbb{R}^3 \to \mathbb{R}$ are constructed[1], and elaborates on how they can be used to represent SDFs and surface normals.

[1]Please refer to [31, 32, 33] for a more in-depth treatment.



Fig. 4: Spline primitives provide an efficient and automated way of obtaining smooth approximations to the SDF of any input mesh. The voxel density N per axis determines the amount of smoothing, with larger numbers providing closer (but sharper) approximations.

1) Cubic Splines in 3D. In 1D, standard cubic splines are built as a sum of twice-differentiable polynomial basis functions $u : \mathbb{R} \to \mathbb{R}$ of the following form [31]:

$$u(t) = \begin{cases} 4 - 6|t|^2 + 3|t|^3 & \text{for } |t| \leq 1 \\ (2 - |t|)^3 & \text{for } 1 \leq |t| \leq 2 \\ 0 & \text{for } 2 \leq |t| \end{cases} \tag{7}$$

Given the values $\{f_i\}_{i=0}^{N}$ of a function $f : [a,b] \to \mathbb{R}$ evaluated at the corners of N intervals splitting $[a,b]$ into equal pieces of length h each, the cubic spline interpolation in 1D is given by the sum $s(t) = \sum_{k=1}^{N+3} c_k u_k(t)$, using a superposition of $N+3$ shifted basis functions defined as $u_i(t) = u(\frac{t-a}{h} - k + 2)$. The coefficients c_k are in turn obtained by solving the linear system defined by the equations $s(t_i) = f_i$.

Given the values $\{f_{i,j,k}\}_{i,j,k=0}^{N_x,N_y,N_z}$ of a function $f : \mathbb{R}^3 \to \mathbb{R}$ evaluated at the corners of a 3D voxel grid with (N_x, N_y, N_z) voxels along each axis, a 3D spline interpolation is obtained as $s(x,y,z) = \sum_{k_x=0}^{N_x+3} \sum_{k_y=0}^{N_y+3} \sum_{k_z=0}^{N_z+3} c_{k_x,k_y,k_z}\ u_{k_x}(x) u_{k_y}(y) u_{k_z}(z)$, using products of the form $u_{k_x}(x) u_{k_y}(y) u_{k_z}(z)$ to define 3D basis functions from 1D basis functions. The coefficients c_{k_x,k_y,k_z} are again obtained by solving the linear system defined by the equations $s(x_i, y_j, z_k) = f_{i,j,k}$.

An important property of spline interpolation is that, because the basis function $u(t)$ has compact support ($u(t) = 0$ for $t \notin [-2,2]$), once the coefficients are obtained, evaluating the spline at arbitrary points is very fast and memory efficient. In 3D, only 64 of the terms in the sum $s(x,y,z)$ are nonzero for any point (x,y,z), and in general there are only 4^D nonzero terms for a spline in D dimensions. Therefore, one only needs to identify the indices of the 64 nonzero terms via integer rounding operations, after which they can be evaluated and summed with little computational cost.

2) Using 3D Splines to Represent SDFs and Normals. The ground truth signed distance and surface normal functions defined by a mesh can be evaluated at any 3D point $\boldsymbol{x}$ by:

- Finding the closest point $\text{proj}(\boldsymbol{x})$ to $\boldsymbol{x}$ on the mesh surface.
- Performing an occupancy check to determine whether $\boldsymbol{x}$ is inside or outside the mesh as $\text{sign}(\boldsymbol{x}) \in \{-1, 1\}$.
- Computing the SDF via $\phi(\boldsymbol{x}) = \text{sign}(\boldsymbol{x}) \|\boldsymbol{x} - \text{proj}(\boldsymbol{x})\|$ and the surface normal via $\boldsymbol{n}(\boldsymbol{x}) = \text{sign}(\boldsymbol{x}) \frac{\boldsymbol{x} - \text{proj}(\boldsymbol{x})}{\|\boldsymbol{x} - \text{proj}(\boldsymbol{x})\| + \varepsilon}$.

Evaluating the ground truth SDF and surface normal functions (that are only once-differentiable) along the corners of a voxel grid and performing a 3D cubic spline interpolation gives a twice-differentiable approximation (where the amount of smoothing is controlled by the voxel grid density). The reason surface normals are not obtained from the gradient of the SDF interpolation is because they also need to be twice-differentiable for ensuring the smooth differentiability of the entire simulation pipeline.

IV. Methods: Contact Manifold Construction

Given the previously described primitive-based and spline-based approaches that provide an efficient way to represent SDFs of complex surfaces, the next step is using them to create contact manifolds between colliding surfaces in a differentiable and vectorizable manner. In the proposed routine, vertices and edges of a mesh collide with an SDF. This happens in three consecutive stages: i) broad-phase, ii) narrow-phase, and iii) blending-phase. In the broad-phase, all vertices and edges that are guaranteed to lie further away than a given distance threshold from the opposing surface are identified and filtered via checks that are more efficient than running the subsequent narrow-phase computations on them. The narrow-phase is where the actual vertex-SDF (V-SDF) and edge-SDF (E-SDF) collisions happen for those edges and vertices that were not filtered. Finally, in the blending-phase, information such as penetration depths, contact normals and Jacobians associated with all V-SDF and E-SDF contacts are fused together using precomputed group IDs based on a convex decomposition, to significantly reduce the total number of contact points that will be passed to the subsequent contact dynamics stage of the overall simulation pipeline. The following sections elaborate on these proposed broad-phase, narrow-phase, and blending-phase routines. Although broad-phase comes before narrow-phase in the overall pipeline, it is covered after narrow-phase for better exposition.

A. Preparing SDF and Mesh Representations

Before delving into the details of the proposed collision detection pipeline, let us first describe step-by-step how the collision meshes and collision SDFs are obtained starting from an arbitrary input mesh:

- The input mesh is processed to obtain the collision SDF. For the primitive-based approach, this is done either through modelling by hand or using an automated decomposition approach such as the method of [25]. For the spline-based approach, this is done by computing ground truth SDF and surface normal values with respect to the input mesh and solving for the spline coefficients.
- Once the collision SDF is obtained, it is converted into an iso-surface mesh using marching cubes. The conversion from the input mesh to the collision SDF and then back into an iso-surface mesh happens because the collision SDF does not have to exactly match the input mesh (since it is smoothed to be twice-differentiable).
- The iso-surface mesh is processed with the re-meshing method of [6] to obtain a reduced number of faces that have high isotropy and in a way that the edges and corners align well with the sharp features of the underlying geometry. This results in the collision mesh.

Since the collision mesh is obtained from simplifying the iso-surface of the collision SDF, these two representations can differ slightly. If it is desirable to obtain an exact match between them, one first runs the proposed collision detection pipeline to obtain contact points on the collision mesh, and

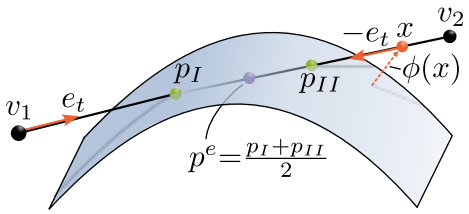


Fig. 5: Intersection points p_I and p_{II} between the SDF $\phi(x)$ of one surface and the edge $v_1 - v_2$ of the other surface are determined via sphere tracing. At every sphere tracing step, the current intersection point candidate x is moved towards the SDF by $\pm[\![\phi(x) > 0]\!]\phi(x)e_t$. The $[\![\phi(x) > 0]\!]$ term ensures that if v_1 or v_2 already lie inside the surface, they are not moved by sphere tracing.

then projects these contact points $\boldsymbol{p}$ onto the corresponding collision SDF isosurface via $\text{proj}(\boldsymbol{p}) = p - \phi(\boldsymbol{p})\boldsymbol{n}(p)$.

B. Narrow-phase

Narrow-phase is where contact points, penetration depths, and contact normals are computed for those V-SDF and E-SDF collisions that are not filtered during broad-phase. V-SDF collisions amount to treating every vertex as a contact point and evaluating the SDF and surface normal functions at these points to assign a penetration depth and a contact normal. E-SDF collisions involve selecting a suitable intersection point among all points on an edge, which is handled via sphere-tracing. Given the straight-forward nature of V-SDF collisions, the rest of this section focuses on E-SDF contacts.

1) Finding Edge-SDF Intersection Points. Points $\boldsymbol{e}$ on an edge between corners $\boldsymbol{v}_I, \boldsymbol{v}_{II} \in \mathbb{R}^3$ can be parameterized as $\boldsymbol{e}(\alpha) = \boldsymbol{v}_I + \alpha \boldsymbol{e}_t$ with $\boldsymbol{e}_t = \frac{\boldsymbol{v}_{II} - \boldsymbol{v}_I}{\|\boldsymbol{v}_{II} - \boldsymbol{v}_I\|}$ and $0 \leq \alpha \leq \|\boldsymbol{v}_{II} - \boldsymbol{v}_I\|$. This means the maximal penetration point between the edge e and any SDF ϕ can hypothetically be found via $\min_\alpha \ \phi(\boldsymbol{e}(\alpha))$ subject to $0 \leq \alpha \leq \|\boldsymbol{v}_{II} - \boldsymbol{v}_I\|$. One can then obtain smooth gradients through this constrained optimization problem by employing log barrier smoothing and applying the implicit function theorem to the resulting KKT conditions [15, 17]. That being said, there are a number of pathologies associated with employing such an edge-SDF collision routine:

- The optimization problem is non-linear, meaning the solution and its gradients obtained via implicit differentiation would still exhibit jumps when the problem switches from one local optimum to another as parameters are varied.
- Unrolling the solver steps (e.g. Newton's method in 1D) and applying automatic differentiation is also not an option since it is computationally expensive. This is because: i) computing Hessians is expensive, and ii) the number of Newton iterations needs to be large (empirically ≥ 10).
- As shown in [24], such procedures are inefficient for vectorization, because a vectorized iterative solver applied to a batch of problems will only finish as quickly as the slowest-converging problem in the batch.

2) Collision Detection with Sphere Tracing. Therefore, we devise a custom edge-SDF collision routine based on sphere-tracing [34]. Given an initial point $\boldsymbol{p}_0$ and a ray direction $\boldsymbol{n}$, sphere-tracing is an algorithm to find ray-SDF intersections (widely considered to be the most efficient way of doing so even for very complex surfaces [35]). It proceeds via updates of the form $\boldsymbol{p}_{k+1} = \boldsymbol{p}_k + [\![\phi(x) > 0]\!]\phi(x)\boldsymbol{n}$. The proposed edge-SDF routine then proceeds as follows:

- Sphere trace the edge corners $\boldsymbol{v}_I$ and $\boldsymbol{v}_{II}$ along $\boldsymbol{e}_t$ and $-\boldsymbol{e}_t$ respectively to obtain $\boldsymbol{p}_I$ and $\boldsymbol{p}_{II}$. We have empirically found three iterations to be more than sufficient for convergence.
- Clip $\boldsymbol{p}_I$ and $\boldsymbol{p}_{II}$ within the edge boundaries via a soft-clip operation, and return the midpoint $\boldsymbol{p}^e = (\boldsymbol{p}_I + \boldsymbol{p}_{II})/2$.

This process is illustrated in Figure 5. Naturally, it is guaranteed to succeed only in cases when an edge penetrates the SDF only once or never. We argue that this is sufficient, because if there exists an edge that penetrates an SDF more than once, this indicates that the mesh tessellation is not at a density adequate for the complexity of the surfaces involved in the collision, and should hence be increased.

After the E-SDF intersection point $\boldsymbol{p}^e$ is found using the proposed sphere-tracing routine, it is treated as a contact point and the SDF and surface normal functions are then evaluated to assign penetration distances and contact normals.

C. Broad-phase

1) Narrow-phase Modifications to Accommodate Differentiable Broad-phase. The goal of the broad-phase is to identify those V-SDF and E-SDF collisions that are guaranteed to produce contact points with a penetration distance larger than a distance threshold $d_{\text{max}} \geq 0$, so that they can be omitted from the subsequent narrow-phase for efficiency. The reason we do not set $d_{max} = 0$ as default is because it is preferable to devise contact models that can exert forces and return gradients at a distance [36, 37, 20], an application that the proposed collision detection pipeline also aims to facilitate. To be able to omit collisions further than $d_{\text{max}} \geq 0$ in a manner that preserves twice-differentiability, we define the following mollification operations to modify contact information returned by the narrow-phase:

- Employing a transition function $f_t : \mathbb{R} \to [0, 1]$ (e.g. a piecewise polynomial) that is non-increasing, twice-differentiable, and satisfies $f_t(d) = 0$ for $d > d_{\text{max}}$ and $f_t(d) = 1$ for $d \leq d_{\text{max}} - \varepsilon$ for a given small bandwidth $\varepsilon \geq 0$.
- Redefining the penetration depth d_c of a contact point $\boldsymbol{p}$ as $d_c(\boldsymbol{p}) = f_t(\phi(\boldsymbol{p}))\phi(\boldsymbol{p}) + [1 - f_t(\phi(\boldsymbol{p}))]d_{\text{max}}$ such that d_c saturates to the value d_{max} with increasing SDF $\phi(\boldsymbol{p})$.
- Redefining the contact normal as $\boldsymbol{n}_c(\boldsymbol{p}) = f_t(\phi(\boldsymbol{p}))\ \boldsymbol{n}(\boldsymbol{p})$, and the contact Jacobian as $\mathbf{J}_c(\boldsymbol{p}) = f_t(\phi(\boldsymbol{p}))\ \mathbf{J}(\boldsymbol{p})$, such that contact points that are further away than d_{max} from the opposing surface do not exert any contact force (assuming the subsequent contact dynamics solver processes $d_c, \boldsymbol{n}_c, \mathbf{J}_c$ in a way that properly handles these modifications).

2) Differentiable Broad-phase. The main enabler of the broad-phase routine is a voxel grid (constructed in the body frame of the surface represented with an SDF) where every voxel stores the value of the SDF function evaluated at its center point. Reading values from this voxel grid is a very fast $\mathcal{O}(1)$ operation. Furthermore, the voxel grid does not need to be duplicated when the contact manifold generation routine is vectorized (since GPUs allow parallel reads from the same memory address), which means storing the voxel-grid does not constitute a memory bottleneck for massive vectorization.

Let $\boldsymbol{p}$ be a 3D point that is Δd away from the center $\boldsymbol{p}_c$ of the voxel within which it falls, with the associated SDF values $\phi(\boldsymbol{p})$ and $\phi(\boldsymbol{p}_c)$ (where $\phi(\boldsymbol{p}_c)$ is already computed and stored with the voxel grid). As shown in Fig.6, the inequality

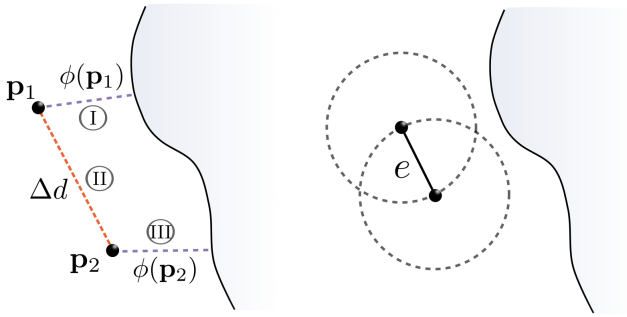


Fig. 6: Left: Among all curves that start at a point $\boldsymbol{p}_1$ and end on a surface, the shortest one is the line segment with length $\|\boldsymbol{\phi}(\boldsymbol{p}_1)\|$ and direction $\nabla\boldsymbol{\phi}(\boldsymbol{p}_1)$. For two points $(\boldsymbol{p}_1,\boldsymbol{p}_2)$ with distance Δd in-between, this implies that $\|\boldsymbol{\phi}(\boldsymbol{p}_1)\| \leq \|\boldsymbol{\phi}(\boldsymbol{p}_2)\| + \Delta d$ holds (since the left side is the length of curve I which is the shortest, and right side is the length of curves II-III concatenated). This property can be used to pre-compute SDF values on a voxel grid and use them to filter vertices in broad-phase. **Right**: An edge with endpoints $(\boldsymbol{p}_1,\boldsymbol{p}_2)$ and length l cannot collide with a surface as long as $\min(\boldsymbol{\phi}(\boldsymbol{p}_1),\boldsymbol{\phi}(\boldsymbol{p}_2)\ ) \geq l$, which can be combined with the previous property (used for vertex filtering) to filter edges in broad-phase.

$\boldsymbol{\phi}(\boldsymbol{p}) + \Delta d \geq \boldsymbol{\phi}(\boldsymbol{p}_c)$ always holds. Given the radius r of the circumsphere of each voxel, this means $\boldsymbol{\phi}(\boldsymbol{p}_c) - r$ can be used as a lower-bound for $\boldsymbol{\phi}(\boldsymbol{p})$. Therefore V-SDF collisions can be filtered by finding the voxel a vertex falls into and checking whether $\boldsymbol{\phi}(\boldsymbol{p}_c) - r \geq d_{\max}$ holds. If it does, the V-SDF collision is filtered away and does not participate in the narrow-phase, and instead automatically gets assigned $(d_{\max}, \mathbf{0}, \mathbf{0})$ as its penetration depth, contact normal and Jacobian.

Let $\boldsymbol{e}$ be an edge with length l and endpoints $(\boldsymbol{p}_1,\boldsymbol{p}_2)$ with corresponding SDF values $(\boldsymbol{\phi}(\boldsymbol{p}_1),\boldsymbol{\phi}(\boldsymbol{p}_2))$. Again using the same arguments from Fig.6, for any point $\boldsymbol{p}$ on the edge, $\boldsymbol{\phi}(\boldsymbol{p}) + l \geq \boldsymbol{\phi}(\boldsymbol{p}_1)$ and $\boldsymbol{\phi}(\boldsymbol{p}) + l \geq \boldsymbol{\phi}(\boldsymbol{p}_2)$ both hold. As in the V-SDF case, this means $\min(\boldsymbol{\phi}(\boldsymbol{p}_1),\boldsymbol{\phi}(\boldsymbol{p}_2)) - r - l$ can be used as a lower-bound for $\boldsymbol{\phi}(\boldsymbol{p})$, which can again be compared to $d_{\max}$ to filter away E-SDF collisions and automatically assign $(d_{\max}, \mathbf{0}, \mathbf{0})$ as the resulting narrow-phase output.

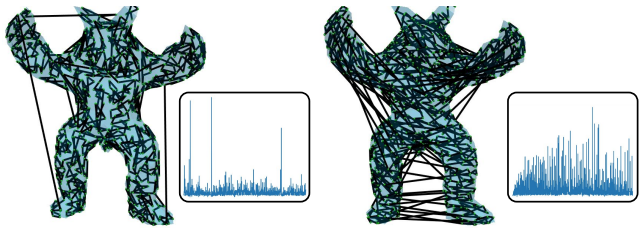


Fig. 7: Vertex re-ordering using a Hilbert curve for serialization, to accelerate broad-phase filtering. The lines connect subsequent vertices. The plots in turn show distances between subsequent vertices in the y axis, and indices themselves in x axis. It can be seen that re-ordering significantly reduces jumps between the 3D positions of vertices that have subsequent indices.

3) Vertex and Edge Reordering to Optimize Memory Access. A GPU organizes threads into groups of 32 called warps, and all 32 threads in a warp progress through kernel instructions in lock step fashion as specified by the Single-Instruction Multiple-Threads (SIMT) paradigm [38]. This means if a kernel contains a control flow branch (e.g. an if/else statement) that is followed by N threads in the warp, the remaining $32-N$ threads sit idle (a phenomenon named "warp divergence"). Minimizing warp divergence means minimizing idle time, making vectorized code run faster.

For the proposed broad-phase and narrow-phase combination, warp divergence is minimized if one maximizes the number of 32 consecutive (i.e. in terms of index order in the array that stores them) vertices/edges that are all accepted or rejected together by broad-phase filtering. Achieving this in turn requires ordering vertices/edges in an array such that if two vertices/edges are close together in terms of 3D Euclidean distance, then their indices are also close together. This is achieved by re-ordering vertices/edges on a mesh based on the traversal order of their projections on a 3D space-filling curve, a standard process (often called "serialization") employed in file sequencing to speed up queries [39]. The proposed framework supports serialization using Z-order and Hilbert curves, building on implementations provided by [40, 41].

D. Blending-Phase

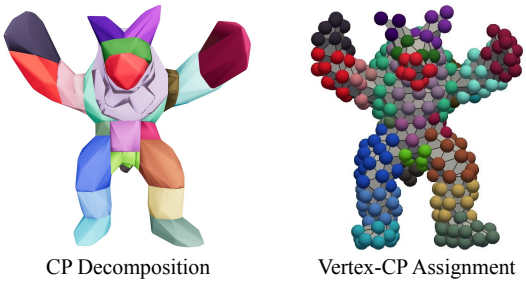


Fig. 8: Left: visualizations of convex primitive (CP) decomposition outputs. **Right**: vertex CP group ID assignments. Vertices (on the right) that are assigned the same CP group ID are plotted with the same color as the corresponding CP itself (on the left).

1) Convex Primitive Assignment. In preparation for contact blending, meshes of both surfaces are processed via a convex decomposition algorithm [9], to obtain N_1 and N_2 convex primitives (CP) respectively. Every CP has an associated list of vertices and edges assigned to it. Vertices are assigned to the CP that they lie inside of, while edges are assigned to every CP they intersect (since meshes are constructed via a computational geometry algorithm [6] to be as isotropic as possible, majority of edges are contained within a single CP and only a very small minority intersects two neighboring CPs). The overall goal of the blending-phase is to reduce the $V_1+E_1+V_2+E_2$ number of contact points to N_1+N_2.

2) Contact Blending (Mechanism). The proposed (optional) contact blending routine works as follows:

- Let the index $i \in \{1,\cdots,K\}$ run over all vertex and edge contact points that belong to the same CP, with coordinates $\{\boldsymbol{p}_{c,i}\}_{i=1}^{K}$ and penetration depths $\{d_{c,i}\}_{i=1}^{K}$.
- A softmax operator is applied on $\{d_{c,i}\}_{i=1}^{K}$ to obtain blending weights $\{w_i\}_{i=1}^{K}$.
- These weights are used to blend all contact information such as penetration depths, surface normals (which are normalized after blending), and contact Jacobians.

3) Contact Blending (Justification). Collisions between two convex polytopes can in most configurations be resolved

by identifying the single unique pair of witness points with maximal penetration in between. This approach only becomes problematic within the neighborhood of those pathalogical configurations where a unique pair of witness points with maximal penetration cannot be identified, such as the case of two boxes stacked on top of another with their contacting faces lying perfectly parallel (where there are infinitely many witness point pairs that have the same maximal penetration depth in-between). In such cases, the standard approach is employing a polygon clipping algorithm to explicitly construct the polygon enclosing the area of intersection between two mesh faces touching each other, and use the corners of this polygon as the contact manifold (possibly further selecting a subset of four points that span maximal area) [13, 5].[2]

Such pathological cases that require a contact manifold are characterized by the property that the points on the manifold have nearly identical penetration depths and contact normals. Therefore, it is physically plausible to assume that the contact forces acting on these points are also nearly identical (both in terms of magnitude and direction). In fact, under the standard dissipative spring-damper based models of compliant contact [42, 43, 44, 45, 46] (that have extensive experimental validation and are thus widely used in commercial multibody simulation tools [47]), these contact forces (that only depend on the contact point depth, normal and velocity) would indeed be identical in static configurations. Given that the contact forces $\boldsymbol{f}_i$ enter the Newton Euler equation as $\mathbf{M}\boldsymbol{a}+\boldsymbol{b}=\tau+\sum_i \mathbf{J}_i^T \boldsymbol{f}_i$ (where the index i runs over points in the contact manifold), and that for nearly identical contact forces $\sum_i \mathbf{J}_i^T \boldsymbol{f}_i \approx (\sum_i \mathbf{J}_i^T)\boldsymbol{f}$ holds, one could argue that the benefits of constructing an entire contact manifold rather than a single witness point pair mainly manifest themselves through the way the contact Jacobian is modified (i.e. determining where exactly the common contact force $\boldsymbol{f}$ should act on the body). And since the contact Jacobian of a free body expressed in body frame is $\mathbf{J}(\boldsymbol{p}_i)=[[\boldsymbol{p}_i]_\times \ \ \mathbf{I}] \in \mathbb{R}^{3\times 6}$ (where $\boldsymbol{p}_i \in \mathbb{R}^3$ is the contact point coordinates in body frame and $[\boldsymbol{a}]_\times \boldsymbol{b}=\boldsymbol{A}\times\boldsymbol{b}$), blending contact points as $\sum_i w_i \boldsymbol{p}_i$ correctly aggregates contact Jacobians as $\mathbf{J}(\sum_i w_i \boldsymbol{p}_i)=\sum_i w_i \mathbf{J}(\boldsymbol{p}_i)$. Additionally, since $\sum_i w_i = 1$, the resulting point $\sum_i w_i \boldsymbol{p}_i$ is guaranteed to stay within the convex hull of the contact points $\boldsymbol{p}_i$ being blended.

Note that the proposed blending approach essentially constitutes an inversion of the standard convex primitive based paradigm. In the standard paradigm, convex primitives are first identified and then this decomposition is used to obtain a small number of contact points (since one gets a single contact point per CP pair using the GJK/EPA routine); whereas in the proposed approach contact points are first identified and the convex decomposition is only then utilized to reduce the number of contact points via blending.

V. Experiments

1) Expressivity of the XPSQ Primitive. The top part of Fig.9 illustrates how a cup can be represented with only 3 XPSQ primitives (i.e. one cylinder subtracted from another, combined with a handle obtained by sweeping a box along a spline). In contrast, convex decomposition using CoACD [9] results in 36 primitives, which, in the absence of a differentiable broad-phase routine, is highly inefficient. The bottom part shows how an armadillo can be efficiently represented with 18 SQs, obtained using the method of [25].

[2]Note that the proposed sphere tracing based E-SDF contact routine serves an analogous purpose to polygon clipping.

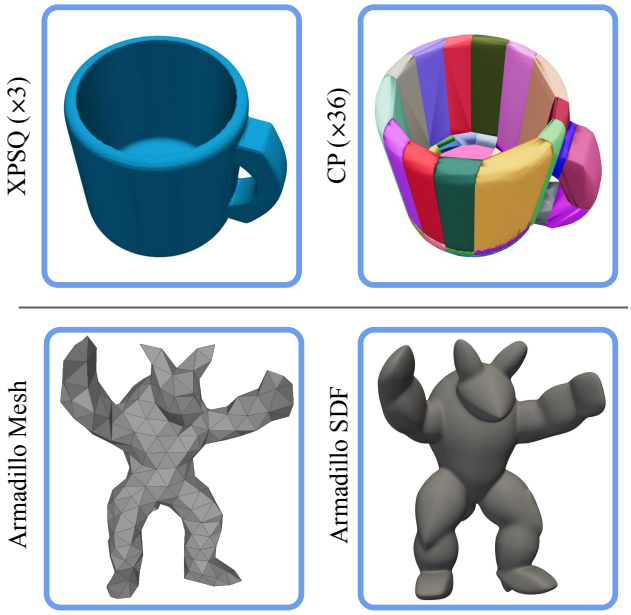


Fig. 9: Examples that demonstrate the expressivity of XPSQs.

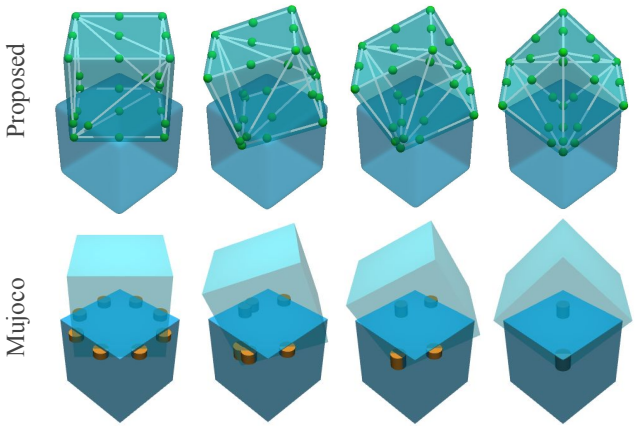


Fig. 10: An example illustrating the behavior of the contact manifold.

2) Behavior of the Contact Manifold. Fig.10 illustrates the qualitative behavior of the contact manifold compared to the polygon clipping routine employed in Mujoco [1], on an example with two boxes stacked on top of each other. It can be seen that polygon clipping results in non-smooth behavior of the contact points, whereas the proposed edge-SDF contact points adapt smoothly to the surface, adequately sampling the volume of intersection between the colliding surfaces. From this perspective, the proposed routine can be interpreted as a middle ground between polygon clipping and the hydroelastic contact model [44], without needing to explicitly re-mesh the intersection volume like the latter (which is a non-differentiable and non-vectorizable routine).

3) Computational Efficiency (Narrow-phase). Fig.11 con-

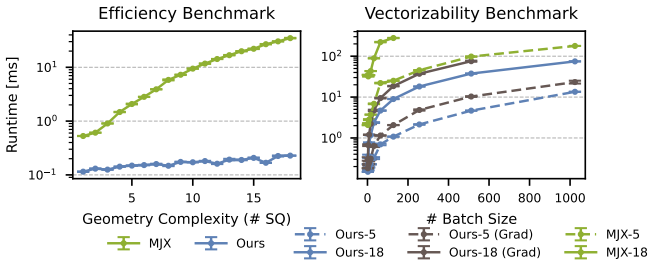


Fig. 11: A benchmark characterizing the computational efficiency of the narrow-phase routine in the absence of broad-phase filtering.

tains plots showing the speed and vectorizability of the proposed narrow-phase routine (implemented in JAX) without broad-phase, compared to the analogous routine of the well-established MJX simulator. The setup involves collisions between two non-convex armadillo geometries across 100 random configurations per-trial. Surfaces are represented with up to 18 SQs each, which is a controlled variable characterizing the effects of geometry complexity on the runtime. The second controlled variable is the vectorization batch size. Notice the logarithmic scale and the order of magnitude improvement compared to MJX.

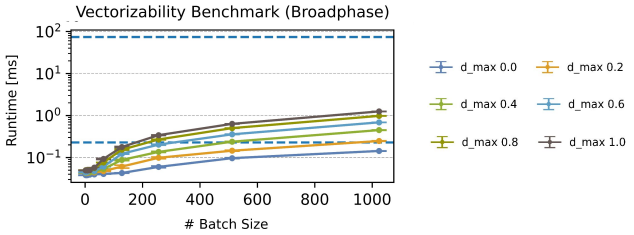


Fig. 12: A benchmark characterizing the computational efficiency of the combined broad-phase and narrow-phase routines.

4) Computational Efficiency (Broad-phase). Fig.12 characterizes the vectorizability of the combined broad-phase and narrow-phase routines for increasing $d_{\max}$ values used for filtering, using the exact same armadillo geometry as the previous section. Since broad-phase filtering requires an if/else branch to be placed at the beginning of the narrow-phase routine for early termination, this experiment uses a separate implementation of both phases in NVIDIA-Warp [48] (which allows kernels containing branching code to be run without triggering a JIT recompilation unlike JAX). In this experiment, every batch with batchsize B is constructed by translating one armadillo away from the other along the x axis by B different linearly spaced distances between $[1,2]$. For reference, the axis aligned bounding-box of the armadillo geometry has an extent of 1.38 units along the x axis, meaning every batch starts from half intersecting bounding boxes (of the two armadillos in collision), and ends with non-intersecting bounding boxes. The horizontal dashed lines in the plot show the runtime of the narrow-phase routine without broad-phase (from the previous section) for batch sizes 1 and 1024, for ease of reference. It can be seen that broad-phase filtering brings another order of magnitude improvement in runtime.

VI. Limitations and Conclusion

We have presented a collision detection routine that is constructed from scratch with smooth differentiability and vectorizability in mind. This is enabled by three contributions: i) expressive and efficient SDF representations, ii) broad-phase and narrow-phase routines to implement V-SDF and E-SDF collisions, iii) a blending-phase that merges contact points together using convex decomposition based groupings. The main limitation is robotics and control experiments that characterize the usefulness of the gradients obtained from the proposed method, which constitutes the most immediate direction for future work.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033. 1, 7

[2] R. Tedrake and the Drake Team, "Drake: Model-based design and verification for robotics," 2019. 1

[3] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," 2016. 1

[4] K. Erleben, "Methodology for assessing mesh-based contact point methods," *ACM Transactions on Graphics (TOG)*, vol. 37, no. 3, pp. 1–30, 2018. 1

[5] D. Gregorius, "Robust contact creation for physics simulations," in *GDC*, 2015. 1, 7

[6] W. Jakob, M. Tarini, D. Panozzo, and O. Sorkine-Hornung, "Instant field-aligned meshes," in *ACM Transactions on Graphics (Proceedings of SIGGRAPH ASIA)*, vol. 34, no. 6, Nov. 2015. 1, 4, 6

[7] E. Corman and K. Crane, "Rectangular surface parameterization," *ACM Trans. Graph.*, vol. 44, 2025. 1

[8] S. Oh, X. Yuan, X. Wei, R. Shi, F. Xiang, M. Liu, and H. Su, "Pamo: Parallel mesh optimization for intersection-free low-poly modeling on the gpu," in *Computer Graphics Forum*. Wiley Online Library, 2025, p. e70267. 1

[9] X. Wei, M. Liu, Z. Ling, and H. Su, "Approximate convex decomposition for 3d meshes with collision-aware concavity and tree search," *ACM Transactions on Graphics (TOG)*, vol. 41, no. 4, pp. 1–18, 2022. 1, 2, 6, 7

[10] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE Journal on Robotics and Automation*, vol. 4, pp. 193–203, 1988. 1

[11] L. Montaut, Q. L. Lidec, V. Petrik, J. Sivic, and J. Carpentier, "Collision detection accelerated: An optimization perspective," *arXiv preprint arXiv:2205.09663*, 2022. 1

[12] G. Van Den Bergen, "Proximity queries and penetration depth computation on 3d game objects," in *GDC*, vol. 170, 2001, p. 209. 1

[13] D. Gregorius, "The separating axis test between convex polyhedra," in *GDC*, 2013. 1, 7

[14] I. E. Sutherland and G. W. Hodgman, "Reentrant

polygon clipping," *Communications of the ACM*, vol. 17, no. 1, pp. 32–42, 1974. 1

[15] K. Tracy, T. A. Howell, and Z. Manchester, "Differentiable collision detection for a set of convex primitives," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 3663–3670. 1, 5

[16] C. J. Ong and E. G. Gilbert, "Growth distances: New measures for object separation and penetration," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 6, pp. 888–903, 1996. 1

[17] K. Tracy and Z. Manchester, "On the differentiability of the primal-dual interior-point method," 2024. 1, 5

[18] L. Montaut, Q. Le Lidec *et al.*, "Differentiable collision detection: a randomized smoothing approach," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 3240–3246. 1

[19] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, pp. 229–256, 1992. 1

[20] A. Paulus, A. R. Geist, P. Schumacher, V. Musil, and G. Martius, "Hard contacts with soft gradients: Refining differentiable simulators for learning and control," *arXiv preprint arXiv:2506.14186*, 2025. 2, 5

[21] M. Li, Z. Ferguson, T. Schneider, T. Langlois, D. Zorin, D. Panozzo, C. Jiang, and D. M. Kaufman, "Incremental potential contact: Intersection- and inversion-free large deformation dynamics," *ACM Trans. Graph. (SIGGRAPH)*, vol. 39, no. 4, 2020. 2

[22] A. Paulus, A. R. Geist, V. Musil, S. Hoffmann, O. Beker, and G. Martius, "SoftJAX & SoftTorch: Empowering automatic differentiation libraries with informative gradients," *arXiv preprint*, 2026. 2

[23] O. Beker, N. Gürtler, J. Shi, R. Geist, A. Razmjoo, G. Martius, and S. Calinon, "A smooth analytical formulation of collision detection and rigid body dynamics with contact," in *2025 IEEE International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025. 2

[24] O. Beker, A. R. Geist, A. Paulus, N. Gürtler, J. Shi, S. Calinon, and G. Martius, "Smoothly differentiable and efficiently vectorizable contact manifold generation," *arXiv preprint arXiv:2602.20304*, 2026. 2, 5

[25] W. Liu, Y. Wu, S. Ruan, and G. S. Chirikjian, "Marching-primitives: Shape abstraction from signed distance function," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 8771–8780. 2, 3, 4, 7

[26] I. Quilez, "Signed distance functions," 2013. 3

[27] ——, "Smooth minimum," 2013. 3

[28] M. Spivak, *A Comprehensive Introduction to Differential Geometry*, 3rd ed. Publish or Perish, Inc., 1999, vol. 2, ch. Curves In the Plane and In Space (The Serret–Frenet formulas), pp. 34–36. 3

[29] Y. Li and S. Calinon, "From movement primitives to distance fields to dynamical systems," *IEEE Robotics and Automation Letters (RA-L)*, vol. 10, no. 9, pp. 9550–9556, 2025. 3

[30] M. Artin, *Algebra*, 2nd ed. Pearson, 2013, ch. 16 – Galois Theory, pp. 501–502. 3

[31] C. Habermann and F. Kindermann, "Multidimensional spline interpolation: Theory and applications," *Computational Economics*, vol. 30, no. 2, pp. 153–169, 2007. 3, 4

[32] M. S. Floater, "An introduction to spline theory," Lecture Notes, Department of Mathematics, University of Oslo, 2025. 3

[33] M.-J. Lai and L. L. Schumaker, *Spline functions on triangulations*. Cambridge University Press, 2007, no. 110. 3

[34] J. C. Hart, "Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces," *The Visual Computer*, vol. 12, no. 10, pp. 527–545, 1996. 5

[35] K. Crane, "Ray tracing quaternion julia sets on the gpu," 2005. 5

[36] I. Mordatch, E. Todorov, and Z. Popović, "Discovery of complex behaviors through contact-invariant optimization," *ACM Transactions on Graphics (ToG)*, vol. 31, no. 4, pp. 1–8, 2012. 5

[37] T. Pang, H. T. Suh, L. Yang, and R. Tedrake, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Transactions on robotics*, 2023. 5

[38] NVIDIA Corporation, "Cuda programming guide, release 13.2," https://docs.nvidia.com/cuda/cuda-programming-guide/pdf/cuda-programming-guide.pdf, 2026. 6

[39] G. M. Morton, "A computer oriented geodetic data base and a new technique in file sequencing.(1966)," 1966. 6

[40] P.-S. Wang, Y. Liu, Y.-X. Guo, C.-Y. Sun, and X. Tong, "O-CNN: Octree-based convolutional neural networksfor 3D shape analysis," *ACM Transactions on Graphics (SIGGRAPH)*, vol. 36, no. 4, 2017. 6

[41] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer v3: Simpler, faster, stronger," in *CVPR*, 2024. 6

[42] V. Kurtz and A. Castro, "Cenic: Convex error-controlled numerical integration for contact," *arXiv preprint arXiv:2511.08771*, 2025. 7

[43] V. Kurtz, A. Castro, A. Ö. Önol, and H. Lin, "Inverse dynamics trajectory optimization for contact-implicit model predictive control," *The International Journal of Robotics Research*, vol. 45, no. 1, pp. 23–40, 2026. 7

[44] R. Elandt, E. Drumwright, M. Sherman, and A. Ruina, "A pressure field model for fast, robust approximation of net contact force and moment between nominally rigid objects," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE Press, 2019, p. 8238–8245. 7

[45] K. L. Johnson, *Contact mechanics*. Cambridge university press, 1987. 7

[46] K. H. Hunt and F. R. Crossley, "Coefficient of restitution interpreted as damping in vibroimpact," 1975. 7

[47] M. S. Corporation, "Adams (automated dynamic analysis of mechanical systems)." 7

[48] M. Macklin, "Warp: A High-performance Python Framework for GPU Simulation and Graphics," Mar. 2022, nVIDIA GPU Technology Conference (GTC). 8